CrowdSafe

AI-Based Stampede Risk Reporting System — Project Build Summary

1. Project Overview

CrowdSafe is an MCA final-year project that lets citizens report dangerous crowd situations by uploading a photo or short video from their phone, along with their GPS location. An AI model estimates the crowd density in the upload and classifies the risk level. The nearest authority (police station / event control room) is automatically identified and notified through a dashboard, so they can review the report and respond. No fixed CCTV infrastructure is required — the system works anywhere a citizen has a phone.

Core design decisions made so far:

- Single responsive web application (not separate native apps) — one backend, role-based views for Citizen and Authority, works on phone browser and laptop.
- Citizen can upload either a photo or a short video (backend branches processing based on file type).
- Relational database — PostgreSQL — chosen over NoSQL since report/user/authority data is structured and relationship-driven.
- Pretrained AI models (CSRNet / YOLOv8) used for crowd counting — no training from scratch required.
- Nearest-authority matching via the Haversine formula on stored lat/long coordinates.
- Entire stack is free to build: Flask, PostgreSQL, PyTorch, OpenCV, Leaflet.js + OpenStreetMap, free hosting tiers for demo.

2. Current Folder Structure

Structure as created so far. Files marked (planned) exist as empty placeholders or have not been created yet, and will be filled in as the build reaches that stage.

```
CrowdSafe AI-Based Stampede Risk Reporting System/
|-- apps.py (Flask entry point - to be finalized)
|-- struct.txt (dev folder listing reference)
|
|-- backend/
| |-- .env (DB credentials, secrets - see Section 4)
| |-- config.py (Flask config - planned, not yet wired in)
| |-- requirements.txt (installed package versions)
| |-- test_connection.py (DB connection test script - WORKING)
| |
| |-- app/
| | |-- __init__.py (currently empty - see Section 4 notes)
| | |-- database.py (SQLAlchemy engine/session setup - WORKING)
| | |-- models.py (ORM table definitions - WORKING)
| | |
| | |-- routes/ (planned - not yet implemented)
| | | |-- auth_routes.py
| | | |-- report_routes.py
| | | |-- authority_routes.py
| | | |-- analytics_routes.py
| | |
| | |-- services/ (planned - stub files created, empty)
| | | |-- crowd_counter.py
| | | |-- frame_extractor.py
| | | |-- risk_classifier.py
| | | |-- trend_analyzer.py
| | | |-- heatmap_generator.py
| | | |-- geo_matcher.py
| | | |-- notifier.py
| | | |-- rate_limiter.py
| | | |-- task_queue.py
| | |
| | |-- uploads/ (will store citizen photo/video uploads)
| |
| |-- models_pretrained/ (will hold CSRNet / YOLOv8 weight files)
| |-- venv/ (Python 3.12 virtual environment)
|
|-- frontend/ (not yet created)
|-- database/schema.sql (not yet created as standalone file)
|-- docs/project_report/ (not yet created)
```

3. What Each Key File Does

backend/test_connection.py

WORKING

Standalone script used to verify the database setup end-to-end. Imports the engine and Base from database.py and the models module, opens a real connection to PostgreSQL, and calls Base.metadata.create_all() to create every table defined in models.py if it doesn't already exist. This was the script used to confirm the database connection is fully working.

Connects to: app/database.py (engine, Base) and app/models.py (table definitions).

backend/app/database.py

WORKING

Sets up the SQLAlchemy connection to PostgreSQL. Reads DB credentials from .env (POSTGRES_HOST, POSTGRES_PORT, POSTGRES_DB, POSTGRES_USER, POSTGRES_PASSWORD), builds the connection URL explicitly using the psycopg2 driver (postgresql+psycopg2://...), and creates the SQLAlchemy engine, SessionLocal, and Base. The .env file path is resolved explicitly via Path(__file__) rather than relying on auto-discovery, since the project folder name contains spaces, which was breaking automatic .env detection.

Connects to: Reads backend/.env. Exposes engine and Base, used by models.py and test_connection.py.

backend/app/models.py

WORKING

Defines the four database tables as SQLAlchemy ORM classes: User, Authority, Report, and Alert, including foreign key relationships (e.g. a Report belongs to a User, an Alert links a Report to an Authority). This is the single source of truth for the database schema.

Connects to: Imports Base from app/database.py. Table classes are used by every route file once they are built, to query and insert data.

backend/.env

WORKING (auto-managed)

Stores all secrets and environment-specific settings: PostgreSQL credentials (POSTGRES_USER, POSTGRES_PASSWORD, POSTGRES_HOST, POSTGRES_PORT, POSTGRES_DB), JWT secret key, optional SMTP credentials for email alerts, and AI risk-threshold values (DENSITY_LOW_MAX, DENSITY_MODERATE_MAX, etc.). Never committed to Git.

Connects to: Read by app/database.py. Will also be read later by config.py and notifier.py.

backend/app/__init__.py

PARKED

Was auto-generated by the Antigravity IDE's built-in assistant as a full Flask application factory (create_app function) with CORS, JWT, and blueprint registration already wired in. This is ahead of the current build stage and assumes files (config.py's config_by_name, routes/ blueprints) that aren't built yet, so it has been temporarily emptied to avoid import errors while the database and AI pipeline are being built first. The original generated version is saved separately and will be reintroduced once routes/ is actually built.

Connects to: Will eventually import from config.py and app/routes/ once those are implemented.

backend/app/services/*.py

PLANNED

Nine placeholder files for the AI and business-logic pipeline: crowd_counter.py (runs the pretrained model on an image or video frames), frame_extractor.py (samples frames from an uploaded video), risk_classifier.py (turns a

person count into Low/Medium/High), trend_analyzer.py (rate-of-change detection across video frames), heatmap_generator.py (density map overlay image), geo_matcher.py (Haversine nearest-authority lookup), notifier.py (email/in-app alerts), rate_limiter.py (throttles uploads per user), task_queue.py (background processing so uploads don't block the request). None are implemented yet — this is the next major build stage.
Connects to: Will be called from report_routes.py once that route file exists.

backend/app/routes/*.py

PLANNED
Will contain the Flask route handlers: auth_routes.py (register/login), report_routes.py (citizen upload + status), authority_routes.py (dashboard, acknowledge alerts, manage authorities), analytics_routes.py (trend/hotspot data). Not implemented yet.
Connects to: Will import from app/models.py, app/database.py, and app/services/*.py.

4. Issues Resolved While Setting Up the Database

The following problems came up while connecting the backend to PostgreSQL and were resolved in order. Worth keeping a record of these for the project diary / viva, since they reflect real debugging work:

- **Python version incompatibility** — the system had Python 3.14 installed by default, which psycopg2-binary does not yet support. Fixed by creating the virtual environment with Python 3.12 instead.
- **Conflicting auto-generated code** — the IDE's built-in assistant had generated a full Flask app factory in app/__init__.py referencing packages and files (config.py, routes blueprints) that didn't exist yet, causing repeated ModuleNotFoundError failures. Resolved by temporarily emptying the file until the project reaches that build stage.
- **Wrong database driver selected** — SQLAlchemy attempted to use the newer psycopg (v3) driver instead of the installed psycopg2. Fixed by explicitly setting the connection string prefix to postgresql+psycopg2://.
- **.env file not loading** — python-dotenv's automatic file discovery failed silently because the project folder path contains spaces ("CrowdSafe AI-Based Stampede Risk Reporting System"). Fixed by explicitly resolving the .env path using Path(__file__).resolve().parent.parent instead of relying on auto-discovery.
- **Mismatched environment variable names** — the IDE assistant regenerated .env using POSTGRES_* variable names, while database.py was originally written expecting DB_* names, causing the password to silently load as empty. Fixed by aligning database.py to read the POSTGRES_* names actually present in .env.
- **Database name mismatch** — .env pointed to a database (crowdsafe_db) that had not been created in pgAdmin yet (an earlier database, crowd_monitor, had been created instead). Fixed by creating crowdsafe_db in pgAdmin to match .env.

5. Current Status

Component	Status
PostgreSQL installed & running	Done
Database 'crowdsafe_db' created	Done
Backend connects to PostgreSQL (test_connection.py)	Done — verified working
Tables created (users, authorities, reports, alerts)	Done — verified in pgAdmin
Citizen upload: photo + video support	Design decided, not yet coded

Risk level tiers (3 vs 4 levels)	Pending decision
AI crowd-counting pipeline	Not started — next stage
Flask routes (auth/report/authority/analytics)	Not started
Frontend (citizen upload + authority dashboard)	Not started

6. Next Steps (Build Order)

1. Decide risk-level tiers: Low/Medium/High (3) or add a Critical tier (4).
2. Update models.py: rename video_path to media_path, add a media_type column ('image' or 'video') to the reports table.
3. Build crowd_counter.py — load a pretrained model (CSRNet or YOLOv8) and get a person count from a single image.
4. Build risk_classifier.py — turn a person count into a risk level using the thresholds already defined in .env.
5. Build frame_extractor.py logic — only triggered when media_type is 'video', samples frames for crowd_counter.py to process.
6. Test the AI pipeline standalone (run it directly on a sample photo/video from disk) before wiring it into any Flask route — this isolates AI bugs from web/API bugs.
7. Build authority_routes.py and a basic dashboard frontend using manually-seeded test data in the database.
8. Build report_routes.py and the citizen upload frontend (file upload + geolocation capture).
9. Wire in geo_matcher.py (nearest authority), notifier.py (alerts), task_queue.py (background processing), rate_limiter.py.
10. Add analytics_routes.py and polish the UI.

Generated as a project development log for CrowdSafe (MCA project). Reflects progress as of the database setup milestone.